

VC Generation

Overview

Goal

VC generation turns a TAC-like program into an SMT query.

$$\text{sat}(\text{VC}(P, A)) \Leftrightarrow \text{there exists an execution reaching failure of } A$$

Tiny TAC (`ttac`) keeps the semantic core visible:

- explicit basic blocks and terminators
- named Boolean branch and assertion conditions
- scalar definitions, havoc, assume, assert
- maps and map updates
- phi nodes at joins

What The Encoder Consumes

A ttac program gives the VC generator:

- a control-flow graph
- commands in each block
- symbol sorts
- scalar and map expressions
- the assertion-failure query

The encoder produces an SMT formula whose models are bug witnesses.

$$VC(P, A) = \text{Path} \wedge \text{Assumes} \wedge \neg \text{Assert}$$

Unsat means no modeled assertion-failure execution exists.

VCGen Styles

- **SeaHorn-style**: explicit block reachability variables.
- **Boogie-style**: backward safety equations.
- **SeaBMC-style**: control-dependence compiled into data-dependence.

The encodings differ mainly in how they represent path semantics and joins.

Running Example Shape

```
entry:
  x := havoc
  y := havoc
  c := x < y
  if c goto left else right

left:
  a_left := x + 1
  goto join

right:
  a_right := y + 1
  goto join
```

Running Example: Join And Exit

```
join:
  a := phi [left: a_left, right: a_right]
  ok := a > 0
  goto exit

exit:
  assume true
  assert ok
  halt
```

This diamond exercises branch conditions, joins, phi nodes, definitions, and the final failure query.

Reading The VC

The query orientation is:

$\text{SAT} \Rightarrow$ a bug witness exists

$\text{UNSAT} \Rightarrow$ the assertion is proved in the modeled semantics

The main design choice is how to represent path semantics:

- explicit reachability variables
- recursive safety equations
- data-flow gates and ITEs